A1.2 REQUERIMIENTOS PARA MACOS

PROGRAMA EDUCATIVO

Ingeniería en Sistemas Computacionales

NOMBRE DEL PROYECTO

Automatización de mantenimiento preventivo con monitoreo permanente utilizando APIs y scripts

EMPRESA

Infraestructura Digital Predictiva Bitlab, S.A.P.I. de C.V.

PRESENTA

Axel Gabriel Ramírez Ortiz
Matrícula: 122043202 · Grupo: S206

ASESOR UPQ:

Ana Laura Lira Cortes

CO-ASESOR UPQ:

Ivan Peredo Valderrama

ASESORA EMPRESARIAL:

M.T.A. Sandra Lucía de la Fuente Bermúdez

Querétaro, Qro., 29 de octubre del 2026

ÍNDICE

1. OBJETIVO DE LA ACTIVIDAD

2. DESARROLLO DE LA ACTIVIDAD

- Versiones de macOS
- La consola: zsh y Bash
- Permisos y cuenta de ejecución
- Programación con launchd
- Privacidad, firma y notarización
- Archivos y red

3. RESULTADOS DE LA ACTIVIDAD

4. PENDIENTES Y SIGUIENTES PASOS

REFERENCIAS

1. OBJETIVO DE LA ACTIVIDAD

Esta actividad es la contraparte de la A1.1. Aquí analicé lo que necesita el agente para funcionar en las computadoras Mac: en qué versiones de macOS puede trabajar, qué herramientas del propio sistema puede usar, con qué permisos se ejecuta, cómo se programa con launchd y qué reglas de seguridad de Apple hay que respetar.

Entre las dos actividades se responde la primera pregunta del proyecto. Lo que más me interesaba era que el agente de Mac entregara **los mismos resultados** que el de Windows, porque eso es lo que plantea la hipótesis H1: un solo formato de reporte para los dos sistemas.

Tabla 1. Datos de la actividad.

Concepto	Descripción
Clave de la actividad	A1.2
Fase del protocolo	F1. Requerimientos y arquitectura
Semana	S2 (14 al 18 de septiembre de 2026)
Horas planeadas	20 horas
Producto	Requerimientos macOS
Entregable al que pertenece	E1. Documento de requerimientos y arquitectura (sub-entregable E1.2)

Fuente: Protocolo de estadía profesional, Bitlab-IT (2026).

2. DESARROLLO DE LA ACTIVIDAD

Para esta parte revisé el protocolo, la guía de Apple para crear servicios en segundo plano (Apple, 2023), su documentación de firma y notarización, y los comandos para Mac que ya están en el catálogo inicial de la demostración. macOS tiene reglas de seguridad más estrictas que Windows en varios puntos, así que buena parte del análisis fue entender qué permite el sistema y qué no.

Versiones de macOS

Tabla 2. Versiones de macOS y su nivel de soporte en el proyecto.

Versión	Nivel de soporte	Por qué
macOS 26 Tahoe y la versión publicada en 2026	Soportada, por validar	Son las versiones más recientes. macOS 26 es la última que Apple lanzó para Mac con procesador Intel.
macOS 15 Sequoia y macOS 14 Sonoma	Soportada. Es la versión principal.	Son las más comunes en equipos administrativos y serán la base de las pruebas.
macOS 13 Ventura	Soportada con reserva	La tomé como versión mínima porque desde ella el sistema avisa al usuario de los programas en segundo plano, algo que el agente necesita manejar.
macOS 12 o anterior	No soportada	Ya no recibe actualizaciones de seguridad.

Fuente: Elaboración propia.

Algo que me ayudó mucho es que los scripts son texto que interpreta la consola, así que funcionan igual en una Mac con procesador **Intel**, como la mía, que en una con **Apple Silicon**. No hace falta preparar una versión distinta para cada tipo de procesador.

La consola: zsh y Bash

Desde macOS 10.15 la consola predeterminada es **zsh**, y por eso decidí escribir el agente para zsh. Un detalle que no conocía antes de revisar la documentación es que el Bash que viene en macOS es muy antiguo, la versión 3.2, por temas de licencia (Free Software Foundation, 2022). Para que los comandos del catálogo también funcionen con Bash, voy a evitar las funciones que solo existen en versiones más nuevas.

Igual que en Windows, la regla es no instalar nada. Nada de Homebrew ni paquetes externos en las Mac de los clientes. Afortunadamente, macOS trae casi todo lo que el agente necesita, como se ve en la Tabla 3.

Tabla 3. Herramientas propias de macOS que usará el agente.

Necesidad	Herramienta	Comentario
Enviar datos a la plataforma	curl	Viene en todas las versiones soportadas.

Necesidad	Herramienta	Comentario
Leer las respuestas en JSON	plutil u osascript	jq sería más cómodo, pero no viene en todas las versiones, así que lo descarté.
Uso del procesador	top -l 2 -n 0	Se usa la segunda medición; la primera no es confiable.
Uso de memoria	sysctl, vm_stat y memory_pressure	macOS habla de presión de memoria; se convierte a porcentaje para compararlo con Windows.
Espacio en disco	df -k /	Se revisa el disco del sistema.
Procesos	ps	La demostración ya lo usa para listar los que más consumen.
Tiempo encendido	sysctl -n kern.boottime	
Errores del sistema	log show --last 1h	Es pesado, así que se hará con poca frecuencia.
Actualizaciones pendientes	softwareupdate -l	Es lento y necesita internet; una vez al día.

Fuente: Elaboración propia.

Permisos y cuenta de ejecución

macOS ofrece dos formas de dejar un programa corriendo en segundo plano. Un **LaunchAgent** funciona solo cuando un usuario inicia sesión y con sus permisos; un **LaunchDaemon** arranca con el equipo y trabaja con permisos de administrador. Considero que el agente debe ser un **LaunchDaemon**, porque acciones del catálogo como limpiar temporales o programar un reinicio no se pueden hacer con permisos de usuario. Es la misma lógica que me llevó a proponer la cuenta SYSTEM en Windows.

En el planteamiento inicial del módulo se había pensado usar sudo sin contraseña para ciertos comandos. Al analizarlo vi que no tenía sentido: si el agente ya corre con permisos de administrador, abrir sudo sin contraseña solo agrega un hueco de seguridad. Por eso lo descarté. Los permisos altos se controlan igual que en Windows, con el catálogo cerrado, una clave por equipo, la confirmación del técnico y el modo de simulación.

Programación con launchd

launchd es el encargado de arrancar y vigilar los programas en segundo plano de macOS (Apple, 2023). Igual que en Windows, separé el agente en dos servicios.

Tabla 4. Servicios launchd del agente.

Servicio	Cuándo se ejecuta	Qué hace
com.bitlab.agente.reporte	Cada 5 minutos	Revisa el estado del equipo y envía el reporte a la plataforma.
com.bitlab.agente.comandos	Siempre; launchd lo reinicia si se detiene	Pregunta si hay comandos pendientes, los ejecuta y envía la salida en vivo.

Fuente: Elaboración propia.

Cada servicio se define con un archivo en /Library/LaunchDaemons. Este es el del servicio de comandos:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
 <key>Label</key>          <string>com.bitlab.agente.comandos</string>
 <key>ProgramArguments</key>
 <array>
  <string>/bin/zsh</string>
  <string>/Library/Application Support/Bitlab/agente.zsh</string>
 </array>
 <key>RunAtLoad</key>       <true/>
 <key>KeepAlive</key>       <true/>
 <key>ThrottleInterval</key>  <integer>10</integer>
 <key>StandardOutPath</key>   <string>/Library/Logs/Bitlab/comandos.log</string>
 <key>StandardErrorPath</key> <string>/Library/Logs/Bitlab/comandos.err</string>
</dict>
</plist>
```

Con **RunAtLoad** y **KeepAlive** el servicio arranca al encender la Mac y se vuelve a levantar si se detiene, mientras que **ThrottleInterval** evita que entre en un ciclo de reinicios si hay un error. Para activarlo, revisar su estado o detenerlo se usarán launchctl bootstrap, launchctl print y launchctl bootout. La Figura 1 muestra cómo trabajan juntos los dos servicios.

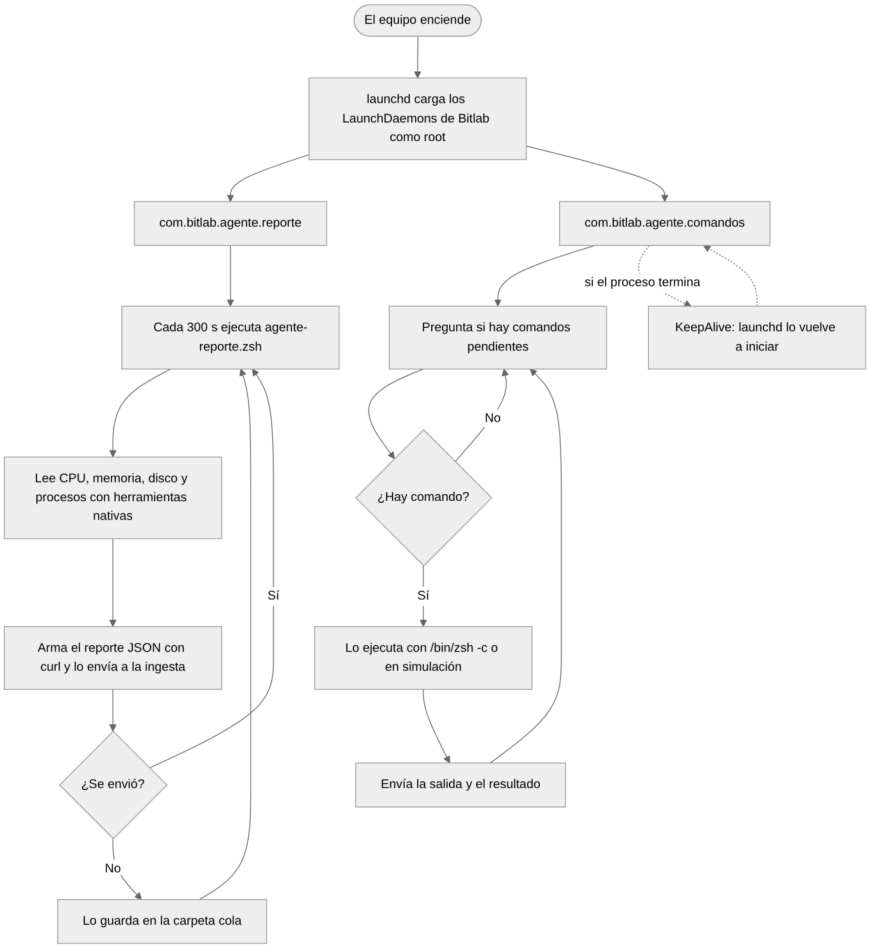


Figura 1. Diagrama de flujo del agente en macOS con sus dos servicios de launchd. *Fuente: Elaboración propia.*

Privacidad, firma y notarización

macOS protege carpetas como Escritorio, Documentos, Descargas, Correo y Safari incluso de los programas que corren como administrador; a esta protección se le conoce como TCC. Al principio pensé que esto iba a ser un problema, pero el agente **no necesita entrar a esas carpetas**: solo revisa datos técnicos, que es justo lo que pide el principio de recolectar solo lo necesario. Así que no se pedirá el acceso total al disco, y ningún comando del catálogo para Mac podrá tocar carpetas de usuario.

Con la firma pasa algo diferente. Los scripts corren a través de zsh, que ya está firmado por Apple, así que el sistema no los bloquea. El instalador sí lo revisa macOS, y para que se instale sin advertencias necesita estar firmado con un certificado de desarrollador y **notarizado**, es decir, enviado a Apple para su revisión. Eso requiere

una cuenta del programa de desarrolladores de Apple, que tiene un costo anual. Para el laboratorio y el piloto propongo que el técnico instale el paquete desde la terminal, y que la firma para producción la decida la asesora externa.

Por último, desde macOS 13 el sistema avisa al usuario cuando se instala algo que corre en segundo plano y lo muestra en los ítems de inicio, desde donde podría apagarlo. La guía de instalación tendrá que explicar este aviso para que nadie desactive el agente por error.

Archivos y red

Los scripts y la configuración irán en /Library/Application Support/Bitlab, y los registros en /Library/Logs/Bitlab. El archivo de configuración, que guarda la clave del equipo, solo lo podrá leer el administrador. En la red se comporta igual que en Windows: solo sale por HTTPS al puerto 443, no abre puertos de entrada y, si no hay internet, guarda los reportes y reintenta esperando cada vez un poco más.



Figura 2. Equipo Mac de desarrollo del practicante. *Fuente: Elaboración propia.*

3. RESULTADOS DE LA ACTIVIDAD

Igual que en Windows, el resultado de esta actividad son los requerimientos funcionales y no funcionales del agente en macOS.

Tabla 5. Requerimientos funcionales del agente en macOS.

Clave	Requerimiento	Descripción	Prioridad
RF-M01	Revisar el estado del equipo	Obtener uso de procesador, memoria, disco, procesos, servicios y tiempo encendido con herramientas nativas.	Alta
RF-M02	Enviar el reporte periódico	Mandar cada 5 minutos el reporte con el mismo formato que Windows.	Alta
RF-M03	Consultar comandos pendientes	Preguntar a la plataforma si hay una ejecución pendiente para su equipo.	Alta
RF-M04	Ejecutar comandos del catálogo	Correr con zsh únicamente el script del catálogo que recibe.	Alta
RF-M05	Simular comandos	Describir lo que haría el comando sin ejecutarlo cuando venga en modo de simulación.	Alta
RF-M06	Enviar la salida en vivo	Mandar la salida por grupos de líneas mientras el comando se ejecuta.	Alta
RF-M07	Informar el resultado	Cerrar cada ejecución con su código de salida y su estado final.	Alta
RF-M08	Guardar reportes sin conexión	Conservar los reportes que no se pudieron enviar y mandarlos al volver la red.	Media

Fuente: Elaboración propia.

Tabla 6. Requerimientos no funcionales del agente en macOS.

Clave	Tipo	Requerimiento	Cómo se comprobará
RNF-M01	Compatibilidad	Funcionar en macOS 13 o posterior con zsh y herramientas nativas, sin Homebrew ni paquetes externos.	Prueba en una Mac limpia.
RNF-M02	Compatibilidad	Funcionar igual en procesadores Intel y Apple Silicon.	Matriz de compatibilidad.
RNF-M03	Disponibilidad	Ejecutarse como LaunchDaemon sin sesión iniciada y reiniciarse si se detiene.	Reiniciar el equipo y revisar el servicio.
RNF-M04	Privacidad	No requerir acceso total al disco ni tocar carpetas protegidas.	Revisión del catálogo y del código.
RNF-M05	Seguridad	Proteger el archivo de configuración para que solo lo lea el administrador.	Revisar los permisos del archivo.
RNF-M06	Interoperabilidad	Generar exactamente el mismo formato de reporte que el agente de Windows (H1).	Validar contra el formato de la A1.5.

Clave	Tipo	Requerimiento	Cómo se comprobará
RNF-M07	Rendimiento	Usar menos del 5 % del procesador en promedio y menos de 2 minutos por ejecución.	Medición en el piloto (H5).

Fuente: Elaboración propia.

Tabla 7. Matriz de compatibilidad por completar en las pruebas.

Equipo	Versión	Procesador	Resultado
Mac del practicante (desarrollo)	macOS sequoia 15.7.9	Intel	Pendiente
Mac de laboratorio de Bitlab-IT	macOS 14 o 15	Apple Silicon	Pendiente
Mac de laboratorio de Bitlab-IT	macOS 26	Por definir	Pendiente

Fuente: Elaboración propia.

4. PENDIENTES Y SIGUIENTES PASOS

- Definir con la asesora externa si Bitlab-IT tendrá la cuenta de desarrollador de Apple para firmar y notarizar el instalador.
- Saber si los clientes administran sus Mac con alguna herramienta central, que permitiría fijar los permisos del agente sin depender del usuario.
- Contar con equipos Mac de laboratorio para las pruebas.
- Comprobar en cada versión soportada cómo leer las respuestas en JSON con plutil u osascript.

Con los requerimientos de los dos sistemas listos, la siguiente actividad es la A1.3, donde se define qué se revisa en cada equipo y cuándo eso se convierte en una alerta.

REFERENCIAS

Apple. (2023). Daemons and services programming guide: Creating launch daemons and agents. Apple Developer Documentation. https://developer.apple.com/library/archive/documentation/MacOSX/Conceptual/BPSystemStartup/

Apple. (2024). Notarizing macOS software before distribution. Apple Developer Documentation. https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution

Bitlab-IT. (2026). Protocolo de estadía profesional (versión 1.0).

Free Software Foundation. (2022). GNU Bash reference manual. https://www.gnu.org/software/bash/manual/